Project Proposal: Plant Disease Classification from Leaf Images using CNNs

Group 18
Marcell Szőkedencsi, Fintan O'Flynn, Mohammed Usman Sarki, Mehak Ezhuvapady Premjit

May 1, 2026

1. Preliminary Domain Knowledge

Plant diseases are a major threat to global food security, with the FAO estimating that 20–40% of annual crop yields are lost to pests and pathogens. In low-resource agricultural settings, diagnosis still relies heavily on visual inspection by trained agronomists, who are scarce relative to demand. Most foliar diseases (e.g., late blight, bacterial spot, powdery mildew, rust) leave characteristic visible symptoms on leaves colored lesions, chlorosis, necrotic patches, mold layers which makes leaf images a natural input modality for automated diagnosis.

Task. We will train a convolutional neural network (CNN) to perform multi-class image classification: given a single RGB photograph of a plant leaf, predict which of 38 (crop, condition) classes it belongs to. Classes span 14 crop species and include both healthy leaves and disease-specific labels (e.g., *Tomato Late Blight*, *Apple Cedar Apple Rust*).

Why this is a sensible ML task. Disease symptoms are localized texture/color patterns that are translation-invariant on the leaf surface exactly what convolutional architectures are designed to exploit. Prior work on the PlantVillage dataset Mohanty et al. (2016) reports 99% test accuracy with deep CNNs, but also documents that these models generalize poorly to real-world field photographs, which is the more interesting open problem.

Prior insight (before looking at the data). Different diseases on different crops can produce visually similar symptoms (e.g., powdery mildew on grape vs. squash). We therefore expect the model to learn shared low-level features (lesion shape, color gradients) across classes rather than purely species-specific representations. This motivates our later choice to test transfer learning, where features pretrained on ImageNet should already encode useful texture priors.

2. Preliminary Data Exploration

Dataset. We will use the PlantVillage dataset (˜54,000 RGB images, 38 classes, originally 256×256). Each image shows a single leaf against a uniform background, captured under controlled lighting.

Features. The features are raw pixel values (3 channels). There are no missing values in the standard sense, but the dataset has known characteristics that act like "hidden missingness": all images are studio-style, so the dataset systematically lacks variation in background, lighting, and leaf orientation that would be present in field deployment.

Class balance. The dataset is moderately imbalanced. The largest class (*Tomato Yellow Leaf Curl Virus*) contains ˜5,300 images while the smallest (*Potato Healthy*) contains ˜150. This is roughly a 35:1 ratio, which we will need to handle in training (Section 3).

Required plots.

1. **Class distribution bar chart** (Figure 1). Confirms the imbalance described above and motivates a class-weighted loss or weighted sampler.
2. **Sample grid + per-class mean image** (Figure 2). The mean images make it visually clear that some classes have very low intra-class variation (controlled background), which raises a generalization concern: the model may latch onto background cues rather than disease features.
3. **t-SNE projection of pretrained ResNet-18 features** on a random subsample of 2,000 images (To be completed further down the line). We use pretrained features rather than raw pixels because raw-pixel embeddings are dominated by background color and yield uninformative blobs. With ResNet features we expect to see partial clustering by crop species first, with disease subclusters within each crop this would confirm the hypothesis from Section 1 that low-level features are shared across the dataset.

Outliers / data quality. Spot-checking reveals a small number of images that are blurry, contain multiple leaves, or are mislabeled. We will not manually clean these; instead we treat them as label noise and rely on regularization (dropout, weight decay, augmentation) to cope.

Confirms / contradicts prior knowledge. The clustering structure expected in t-SNE supports the idea of shared visual features across classes. The class imbalance contradicts the implicit assumption that we can train naively it forces design decisions in preprocessing.

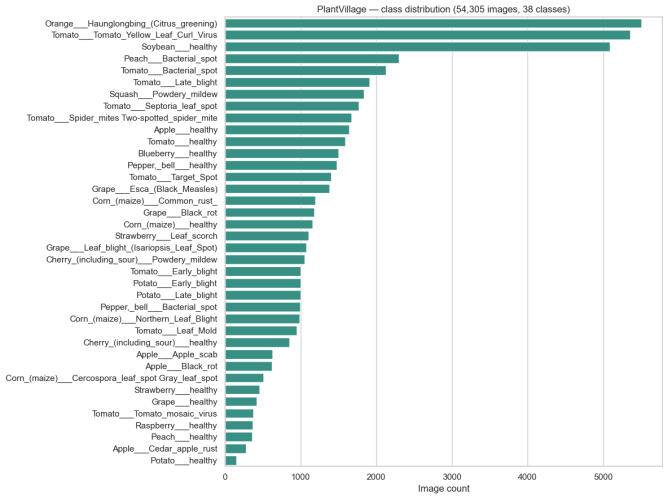

Figure 1: Class distribution across the 38 PlantVillage classes.

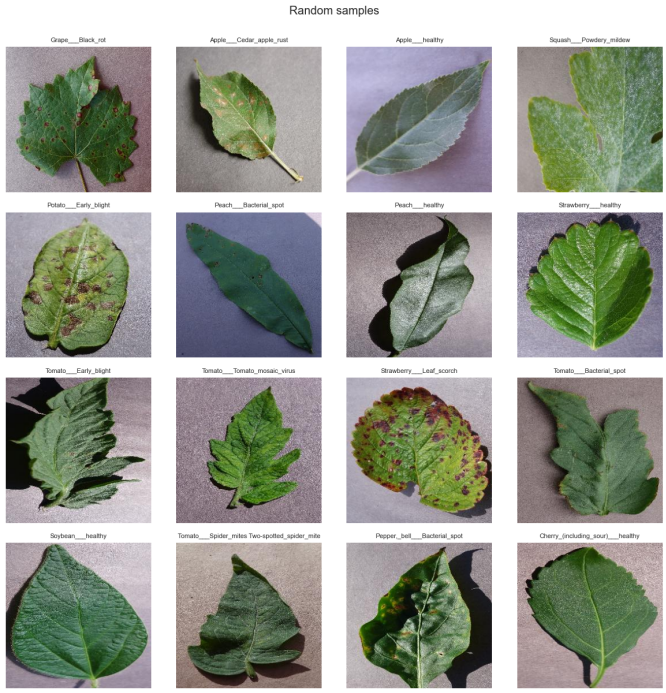

Figure 2: Random samples across 16 classes from the PlantVillage dataset, illustrating visual variation in disease symptoms and the consistent studio-style background.

3. Proposed Preprocessing

Our preprocessing pipeline is driven by three findings from Section 2: (i) the dataset is class-imbalanced, (ii) images are studio-style and may not transfer to field conditions, and (iii) input resolution (256×256) is larger than what most pretrained backbones expect.

Steps.

1. **Resize to 224×224.** Matches the input resolution of ResNet/EfficientNet pretrained on ImageNet, which we use for transfer learning.
2. **Stratified split: 70% train / 15% validation / 15% test.** Stratification preserves the class ratios in each split, which matters for a 35:1 imbalanced dataset. We use a fixed random seed for reproducibility.
3. **Normalization.** For the transfer learning model we use ImageNet channel means/stds (this is required for the pretrained weights to be meaningful). For the from-scratch CNN we compute per-dataset means/stds on the training split only.
4. **Class imbalance handling.** We will use a class-weighted cross-entropy loss (weights) as the default, and compare against a `WeightedRandomSampler` for analysis. We deliberately do **not** oversample by duplication because it interacts poorly with augmentation (the same image gets the same crop multiple times per epoch).
5. **Data augmentation (training only).** Random horizontal flip, random rotation ±15°, color jitter (brightness/contrast/saturation ±0.2), and random resized crop (0.8–1.0 scale). Augmentation directly addresses finding (ii): by perturbing background and orientation in training, we discourage the model from memorizing studio-specific cues.
6. **No PCA / no manual feature extraction.** CNNs learn their own features; reducing dimensionality before a CNN would discard exactly the information the model is designed to use. The baseline (Section 4) is the only model that uses a hand-crafted reduction.
7. **No imputation.** The dataset has no missing values in the tabular sense.

A flowchart of the pipeline (placeholder) will be included in the final repository as `reports/figures/preprocessing_flow.png`, produced in draw.io.

4. Proposed Models and Baseline

Baseline: logistic regression on flattened low-resolution grayscale images. We resize images to 32×32, convert to grayscale, flatten to a 1024-dim vector, and fit a multinomial logistic regression with L2 regularization. We chose this baseline because it is genuinely simple (no spatial reasoning, no nonlinearity beyond softmax) but still uses pixels it gives us a clear floor that any "proper" CNN should beat by a wide margin.

If the CNN's improvement over this baseline is small, that is a strong diagnostic signal that something is wrong (data leakage, overfitting, or label noise).

Full model 1: CNN trained from scratch. A 4-block convolutional network: each block is Conv(3×3)–BatchNorm–ReLU–MaxPool(2×2), with channel widths 32-64-128-256, followed by global average pooling, dropout ($p = 0.3$), and a 38-way linear classifier. This model is included primarily for comparison: it tells us how much of the final performance comes from architecture vs. from pretrained features. We expect it to underperform the transfer model, especially on the rare classes.

Full model 2: Transfer learning with ResNet-18. We initialize from ImageNet-pretrained weights, replace the final FC layer with a 38-way head, and fine-tune. We will compare two regimes: (a) head-only training (all conv layers frozen) and (b) fine-tuning the last residual block plus the head. Fine-tuning the entire network is unnecessary at this dataset size and risks overwriting useful low-level features. We chose ResNet-18 rather than larger variants because it is small enough to train repeatedly during hyperparameter search on a single GPU, while still being expressive enough to reach the published performance ceiling for this dataset.

Training. AdamW optimizer, cosine learning rate schedule with a 1-epoch warmup, batch size 64, up to 30 epochs with early stopping on validation macro-F1 (patience 5). Class-weighted cross-entropy loss. We log all runs with seeds and configs.

Hyperparameter tuning. Random search over: learning rate $\in [10^{-5}, 10^{-2}]$ (log-uniform), weight decay $\in \{0, 10^{-4}, 10^{-3}\}$, dropout $\in \{0.1, 0.3, 0.5\}$, augmentation strength (light vs. heavy color jitter). We chose random over grid search because random search is known to be more efficient when only a few hyperparameters matter (Bergstra and Bengio, 2012), which we expect to be the case here (learning rate dominates). We allocate ˜20 trials per model on the validation split.

5. Proposed Evaluation

Quantitative metrics (predictive performance).

- **Top-1 accuracy** on the held-out test set the standard, but misleading on imbalanced data, so reported alongside:
- **Macro-F1** averages F1 across classes, weighting rare classes equally. This is our *primary* metric for model selection.
- **Per-class precision and recall** to identify classes the model fails on.
- **Confusion matrix** (38×38) to identify systematic confusions, especially between visually similar diseases on the same crop.

Beyond predictive performance.

- **Inference cost:** mean wall-clock time per image on CPU (deployment target) and model size in MB. A model that achieves 99% accuracy but takes 5s/image on a phone

is useless for our user group (Section 6).

- **Train cost:** GPU-hours and energy estimate per full training run, reported transparently.
- **Bias / fairness check:** we report macro-F1 *per crop species*. If one crop is much worse than others, that is a deployment risk we need to flag, not hide behind the average.

Detecting over- and underfitting.

- Plot train and validation loss/accuracy per epoch. A growing gap = overfitting; both flat and high = underfitting.
- Compare train accuracy to test accuracy on the final model. A gap of >5–10 percentage points is a red flag.
- Run the baseline at the same time if the CNN does not clearly beat it, we are likely underfitting (or the task is easier than we thought, which is also useful to know).

6. Model Usage

User group. Smallholder farmers and agricultural extension workers, particularly in regions where access to trained plant pathologists is limited. A secondary user group is hobbyist gardeners.

Use case. A user takes a photo of a suspicious leaf with their phone, uploads it through a simple web app, and receives a top-3 ranked list of likely conditions with confidence scores and a short human-readable description (e.g., "Tomato Late Blight (87% confidence). Common in cool, wet conditions. Consider isolating affected plants.").

Before the model. Image upload → EXIF stripping (privacy) → resize to 224×224 → ImageNet normalization → tensor batch of size 1.

After the model. Logits → softmax probabilities → top-3 class indices → mapped through a JSON lookup to human-readable names (`class_3` → "Tomato Late Blight") → confidence threshold check: if the top-1 probability is below 0.5, the response is "Low confidence consider consulting an expert" rather than a definitive label. We will implement the lookup and threshold logic ourselves.

Deployment scope. We will build a working Streamlit demo running the trained model locally. We will *not* build a production mobile app, on-device inference, or a streaming pipeline those are out of scope but we will discuss what would be needed.

7. Risk Assessment

Threats to project development.

- **Compute bottleneck** (likelihood: medium). Hyperparameter search across two models could exhaust available GPU time. *Mitigation:* cap to 20 trials per model, use

ResNet-18 (small), and run baseline experiments on CPU.

- **Performance ceiling** (likelihood: medium). PlantVillage is so easy that we may saturate at 99%+ test accuracy and have nothing to optimize. *Mitigation:* we will additionally evaluate on a small set of in-the-wild field photographs (e.g., from PlantDoc) to give the project meaningful headroom and a real story to tell.
- **Class imbalance harming rare classes** (likelihood: high but managed). *Mitigation:* class-weighted loss, macro-F1 as primary metric, per-class reporting.

Estimated likelihood of success. High. The task is well-studied, the dataset is clean, and our pipeline uses standard components. Reaching >95% test accuracy on PlantVillage is a low bar; the more interesting question is what the model does on out-of-distribution field images, and we expect to find informative failures there.

Risks of public deployment.

- **False negatives** (predicted healthy when diseased) could delay treatment and cause crop loss direct economic and food-security harm.
- **False positives** (predicted disease when healthy) could trigger unnecessary pesticide application environmental and economic cost.
- **Distribution shift.** The model is trained on studio images and may silently fail on real field photos, where users would most need it. Without explicit communication, users may over-trust it.
- **Liability and substitution.** The system is not a substitute for a plant pathologist. We will surface this prominently in the UI: low-confidence predictions return "consult an expert" rather than a label.

8. Individual Learning Outcomes

- **Marcell Szőkedencsi:** I want to learn how to use Weights & Biases for experiment tracking, because keeping HP search results in a notebook is not going to scale to 40+ runs and I want a workflow I can reuse in future projects.
- **Fintan O'Flynn:** I want to learn how to tune a ML model and do proper analysis on the results. Also very interested on how to use docker during model-deployment and completing a full end-to-end project without the time stress found when doing introduction to machine learning.
- **Mohammed Usman Sarki:** I mainly want to get familiar with properly deploying a model and applying the knowledge gained in Intro to ML in a way that is applicable in the real world.
- **Mehek E.P.:** I want to learn how to deploy an ML model with Streamlit and package it with Docker, because shipping a usable demo is the part of an ML project I have the least experience with and it is what makes the work tangible to non-ML readers.

9. Relation to Grading Specifications

Minimum requirements.

- *Novelty:* we go beyond a vanilla PlantVillage benchmark by (i) explicitly evaluating on out-of-distribution field photographs, (ii) reporting per-crop bias, and (iii) integrating Grad-CAM into the deployed demo. The dataset is well-known, but the framing (deployment-aware, OOD-aware) is not the default treatment.
- *Preprocessing:* stratified split, ImageNet normalization, augmentation pipeline, class-weighted loss all documented, all justified, all reproducible from a config file.
- *Hyperparameters optimized:* learning rate, weight decay, dropout rate, augmentation strength, and (for transfer learning) which layers to unfreeze. Tuned via random search on the validation split.
- Standard requirements (baseline, full model, evaluation metrics, train/val/test split, reproducibility seeds, clean code repo, write-up) are all addressed elsewhere in this proposal.

Grade boosters we plan to attempt.

- Grad-CAM visualizations integrated into the deployed demo, not just a notebook figure.
- Out-of-distribution evaluation on a small set of field-condition images.
- Uncertainty estimation via MC Dropout, used to drive the "consult an expert" fallback.
- Comparison of two backbones (ResNet-18 vs. EfficientNet-B0) under matched compute.
- A short ablation on augmentation strength to quantify its effect on OOD performance.

10. Timeline

Task	W1	W2	W3	W4	W5	W6	W7	W8
Team formation, project idea, repo setup	•							
Literature search, writing the proposal	•							
Data download, EDA, preprocessing pipeline		•	•					
Training the baseline model			•					
Training CNN-from-scratch			•	•				
Transfer learning + hyperparameter tuning				•	•			
Evaluation, confusion matrix, OOD test					•	•		
Grad-CAM, uncertainty, Streamlit demo						•	•	
Poster, report, final writeup							•	•

Table 1: Planned timeline (*Ideally we will follow this but it is just a guideline*). Tasks can run in parallel: e.g., two members work on tuning while the others start on deployment.

11. Member Contributions

- **TBD:** Sections 1, 2; preliminary data exploration plots.
- **TBD:** Sections 3, 4; preprocessing flowchart; model architecture decisions.
- **TBD:** Sections 5, 6; evaluation plan; deployment design.
- **TBD:** Sections 7, 8, 9, 10; risk assessment; grading alignment; timeline.
- All members contributed to the literature search and reviewed the full document before submission.

LLM Usage Statement

Large language models (LLMs) were used as an assistive tool for style and formatting. All design choices of the project and written evaluative content were the authors' full responsibility.

References

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

Sharada P. Mohanty, David P. Hughes, and Marcel Salathé. Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7:1419, 2016.